

Charte de nommage

1. Git

Branches — format `type/scope-description`, tout en kebab-case, sans majuscules ni accents.

- `feat/` : nouvelle fonctionnalité visible ou technique — `feat/skeleton-catalogue`, `feat/endpoint-metadata-interne`
- `fix/` : correction de bug — `fix/vues-counter-updated-at`
- `refactor/` : changement de code sans changement de comportement — `refactor/inline-styles-vers-classes`
- `docs/` : documentation seule — `docs/decision-monolithe-modulaire`
- `chore/` : maintenance (deps, config, CI) — `chore/upgrade-tailwind-v4`
- `test/` : ajout ou correction de tests

Règles complémentaires : une branche = un sujet, durée de vie courte (idéalement < 1 semaine, sinon rebase régulier sur `main`), suppression après merge. Pas de branche perso type `megane-dev` — le nom décrit le travail, pas la personne. Si vous adoptez plus tard des tickets (GitHub Issues), le numéro se met en fin : `feat/filtres-catalogue-42`.

Commits — Conventional Commits strict : `type(scope): description`.

- La description est à l'impératif présent, sans majuscule initiale, sans point final : `add`, `remove`, `fix`, pas `added` ni `fixes`
- Le scope correspond au module touché : `catalogue`, `annonce`, `parcelle`, `agriscore`, `messagerie`, `auth`, `pipeline`, `schema`, `ci`
- Un commit = un changement logique. Si tu écris "and" dans la description, c'est probablement deux commits
- Le corps du commit (optionnel) explique le *pourquoi*, jamais le *quoi* — le diff montre déjà le quoi. Exemple utile pour vous : le commit qui retire `destinataire_id` de la contrainte unique devrait expliquer en corps le bug de conversation dupliquée, pour qu'Ibrahim comprenne sans te demander

Bénéfice concret : avec Conventional Commits, tu peux générer un changelog automatiquement et le préfixe `feat/fix` filtre l'historique en une commande (`git log --oneline --grep="^fix"`) — utile pour la recette et le rapport de stage.

2. Front-end (Next.js)

Composants — PascalCase, un composant par fichier, le fichier porte le nom du composant :

`CardParcelle.jsx` exporte `CardParcelle`. Convention de préfixe par famille :

- **Card*** : cartes de contenu — `CardParcelle`, `CardVendeur`
- **Badge*** : indicateurs de statut — `BadgeStatutFoncier`, `BadgeNouveau`
- **Skeleton*** : états de chargement — `SkeletonCardParcelle`, `SkeletonCatalogue`
- **Empty*** : états vides — `EmptyCatalogue`, `EmptyFavoris`
- **Filtre*** ou **Filter*** (choisis-en un, voir §4) : `FiltreRegion`, `FiltreSurface`

Cette logique de préfixe fait que l'autocomplétion de VS Code te liste toute une famille en tapant trois lettres — précieux quand le nombre de composants grimpe en Phase 3.

Hooks — camelCase, préfixe `use` obligatoire (règle React, pas juste stylistique : le linter s'appuie dessus).

Le nom décrit ce que le hook retourne, pas ce qu'il fait en interne : `useFiltresParcelles` (retourne les filtres et leurs setters), `useAnnonce(id)` (retourne une annonce), `useDebounce(value)`.

Structure de dossiers — kebab-case pour les dossiers, PascalCase pour les fichiers composants :

```
frontend/  
  components/  
    ui/          ← primitives réutilisables (Button, Input, Badge)  
    catalogue/   ← composants spécifiques à l'écran catalogue  
    annonce/     ← composants de la fiche détail  
  hooks/  
  lib/           ← utils purs (formatPrix.js, apiClient.js)  
  styles/        ← globals.css et éventuels modules
```

La distinction `ui/` vs dossiers métier est importante : un composant dans `ui/` ne connaît rien du domaine (pas de prop `parcelle`), un composant dans `catalogue/` peut en dépendre. Ça t'évitera le composant fourre-tout.

CSS — puisque tu migres des styles inline vers des classes, c'est le moment de fixer la règle. BEM allégé :

- Bloc : `.card-parcelle`
- Élément : `.card-parcelle__score`, `.card-parcelle__prix` (double underscore)
- Modificateur : `.card-parcelle--vendue`, `.btn--accent` (double tiret)

Un seul niveau d'élément, jamais `.card-parcelle__header__titre` — si tu as besoin de descendre, c'est le signe qu'il faut un nouveau bloc. Les tokens de `globals.css` en `--akal-*` groupés par catégorie : `--akal-color-foret`, `--akal-color-terracotta`,

`--akal-radius-card`, `--akal-shadow-card-hover`, `--akal-space-md`. Le préfixe `akal-` évite toute collision avec les variables générées par Tailwind v4.

3. Back-end (Django)

Modèles — PascalCase, singulier, sans préfixe d'app : `Parcelle`, pas `Parcelles` ni `AnnonceParcelle`. Le pluriel est géré par `verbose_name_plural` si le français l'exige (`verbose_name_plural = "annonces"`).

Champs — snake_case, avec trois règles précises :

- **Unité dans le nom quand il y a ambiguïté** : `surface_ha`, `prix_mad`, `distance_route_km`. Ça évite le bug classique où quelqu'un stocke des m² dans un champ pensé en hectares. Pas d'unité pour les champs non ambigus (`titre`, `description`)
- **Booléens préfixés** `is_`, `has_` ou `est_` selon ton arbitrage franglais : `is_active`, `has_eau` ou `est_irriguee` — mais un seul préfixe pour tout le projet
- **Dates suffixées** `_at` pour les timestamps techniques (`created_at`, `published_at`), `date_` pour les dates métier (`date_visite`)

Clés étrangères — le nom du champ est le modèle cible en snake_case, sans suffixe `_id` (Django l'ajoute en base automatiquement) : `parcelle = ForeignKey(Parcelle)` donne `parcelle_id` en colonne. Pour les relations multiples vers un même modèle, le rôle prime : `initiateur = ForeignKey(User)`, pas `user1`.

Endpoints DRF — kebab-case, pluriel, ressources imbriquées au maximum d'un niveau :

- `/api/parcelles/`, `/api/parcelles/{id}/`
- `/api/annonces/{id}/photos/` — un niveau d'imbrication OK
- `/api/internal/parcelles/{id}/metadata/` — le préfixe `internal/` isole les endpoints service-to-service (Prefect) des endpoints publics ; ça te permettra d'appliquer une politique d'auth différente (service token) sur tout le préfixe en un seul middleware

Apps Django — snake_case pluriel : `annonces`, `comptes`, `messagerie`, `scoring`. Cohérent avec ta décision monolithe modulaire : une app = un module fonctionnel = une frontière claire, c'est l'argument même que tu as documenté face à la directive "micro-databases" de Baroud.

4. L'arbitrage franglais

C'est le point qui dégénère le plus vite en incohérence sur un projet bilingue. La règle en trois lignes :

1. **Vocabulaire du domaine en français** : `Parcelle`, `Annonce`, `statut_foncier`, `melkia`, `soulaliya`, `type_culture`. Traduire ces termes en anglais ferait perdre le lien avec le métier marocain — "melkia" n'a pas de traduction.
2. **Vocabulaire technique en anglais** : `created_at`, `metadata`, `is_active`, `useDebounce`, `apiClient`. Ce sont des conventions universelles que traduire rendrait bizarres (`cree_le` surprendrait n'importe quel dev qui rejoint le projet).
3. **Jamais de mélange dans un identifiant** : `typeCulture` est correct (mot métier français en camelCase), `cultureType` non (moitié traduit). Le test rapide : chaque mot de l'identifiant appartient-il clairement à la colonne français-métier ou anglais-technique ?

La zone grise, c'est les verbes d'action côté front (`FiltreRegion` ou `FilterRegion` ? `formatPrix` ou `formaterPrix` ?). Ma recommandation : verbes et préfixes techniques en anglais (`format`, `use`, `fetch`, `Filter`), noms métier en français (`Prix`, `Parcelle`, `Region`) — donc `formatPrix`, `FilterRegion`, `useFetchAnnonces`. C'est le compromis le plus lisible et le plus proche de ce que tu écris déjà naturellement.

5. Base de données

Django génère les noms de tables, mais deux points à fixer avec Ibrahim avant les migrations :

- **Tables** : laisser le défaut Django `app_modele` (`annonces_parcelle`) — ne pas surcharger `db_table` sauf raison forte, ça complique les migrations pour rien
- **Contraintes et index nommés explicitement** quand ils portent du sens métier : la contrainte corrigée sur `CONVERSATION` devrait s'appeler `unique_conversation_par_annonce_initiateur` dans le `Meta.constraints`, pas le nom auto-généré illisible. Quand elle pètera en prod avec une `IntegrityError`, le nom te dira immédiatement quoi regarder
- **Clés JSONField** (`metadata`, `sous_scores`) : `snake_case` aussi, même si c'est du JSON — `{"ndvi_moyen": 0.42, "pluviometrie_mm": 380}`. Sans cette règle, tu te retrouves avec un mélange camelCase/snake_case dans le JSON que personne ne pourra requêter proprement en PostgreSQL

6. Documents et livrables

Format `AKAL_<Type>_<Sujet>_v<X>.<ext>`, cohérent avec l'existant :

- `AKAL_Plan_Directeur_v1.docx`, `AKAL_Spec_AgriScore_v1.docx`, `AKAL_Recette_Parcours_v1.xlsx`
- Version entière à chaque validation externe (Baroud), pas de `v1.2_final_FINAL` — les itérations intermédiaires vivent dans l'historique du fichier ou en `v1_draft`

- Diagrammes Mermaid : même sujet que le doc qu'ils illustrent —
`gantt_plan_directeur.mermaid`, `erd_schema_v2.mermaid`